

# Dogo POS Documentation

## Dogo POS — Documentation

A multi-business point-of-sale web app with camera barcode scanning, cloud sync, role-based Admin/Teller accounts, offline sales, and KRA VAT-ready reporting.

**Live app:** <https://allen-otly.github.io/mini-pos/> **Backend:** Supabase project `mini-pos-backend` (id: `uzwomzkzqrpiumtnniik`) **Source code:** <https://github.com/Allen-Otty/mini-pos>

### 1. What this is

Dogo POS is a Progressive Web App (PWA) — it runs entirely in a phone or computer browser, can be “installed” to a home screen like a native app, and works offline. It is **multi-tenant**: any number of separate businesses can sign up, and each business’s data (products, sales, customers, expenses, staff) is completely isolated from every other business on the platform.

#### Core roles

| Role          | Can do                                                                                                                                                                                    |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Admin</b>  | Everything: manage catalog, view Dashboard/Reports, manage Team (create tellers), manage Customers, view & log Expenses, change Settings, sell items                                      |
| <b>Teller</b> | Sell items (Sell tab), view/add Customers, log expenses (e.g. petty cash) but <b>cannot</b> view expense totals/history, <b>cannot</b> see Dashboard, Reports, Catalog, Team, or Settings |

### 2. How VAT works in this app

**Prices you enter are the final, VAT-inclusive selling price** — exactly what the customer pays. VAT is **extracted backward** from that price for KRA record-keeping, never added on top at checkout.

Formula used everywhere in the app:

```
VAT amount = price - (price ÷ (1 + VAT rate))
```

Example: an item priced at **KES 116** with **16% VAT**: - VAT amount =  $116 - (116 \div 1.16) = \text{KES } 16.00$  - Amount excluding VAT = **KES 100.00** - Customer still pays exactly **KES 116** — nothing is added at checkout.

This applies consistently in the Catalog, Cart, Receipt, Dashboard, and Reports.

### 3. Signing up (Admin / new business)

1. Open the app link
2. Tap **“Create Business”**
3. Fill in: Business Name, Business Type (Retail/Wholesale/Supermarket/SME/Hotel/Hardware), Your Full Name, Email, Password

4. Tap **Create Business Account**
5. Check your email for a **6-digit code** and enter it on the Verify screen
6. Once verified, your business and Admin account are created automatically and you're logged in

**Note:** Phone sign-up is visible in the UI but disabled until an SMS provider (e.g. Africa's Talking or Twilio) is connected to the Supabase project — see Section 8.

## 4. Adding Teller (till/attendant) accounts

Only an Admin can create tellers — there is no public teller signup.

1. Log in as Admin → **Team** tab
2. Enter the teller's name, email, and a password you choose for them
3. Tap **Create Teller Account**
4. Give the teller their email + password to log in with

Tellers log in through the normal **Sign In** tab — they never see the “Create Business” flow.

## 5. Daily use

### Selling (Admin or Teller)

1. **Sell** tab → point the camera at a barcode, or type a code manually, or pick from the dropdown
2. If the code isn't in the catalog yet, **Admin** is prompted to add it on the spot (Tellers see “ask an Admin” instead)
3. Adjust quantities with +/- in the cart
4. Optionally attach a Customer
5. Tap **Checkout & Print Receipt** — a printable/PDF-able receipt appears

### Offline selling

If there's no internet connection: - An orange banner appears: “You're offline — sales will save locally and sync when back online” - Checkout still works completely — the sale is queued on the device - Once internet returns, queued sales sync automatically (or tap the blue sync banner to force it)

**Limitation:** offline-added Customers and offline Expense entries are **not** queued for sync — only Sales are queued. Add customers/expenses again once back online if they were entered offline.

### Restocking & low stock

- **Catalog** tab (Admin) → Restock an item → pick item + quantity → Add Stock
- When any item's stock drops to its Reorder Level or below, a red banner appears app-wide for Admins, and the item is listed on the Dashboard

## 6. Data model (for reference)

All tables live in Supabase Postgres, under Row Level Security (RLS) — every query is automatically scoped to the logged-in user's `business_id`, so one business can never see another's data.

| Table                   | Purpose                                                            |
|-------------------------|--------------------------------------------------------------------|
| <code>businesses</code> | One row per registered business (name, type, theme, logo, KRA PIN) |

|                    |                                                                                       |
|--------------------|---------------------------------------------------------------------------------------|
| profiles           | One row per user account, linked to a business, with <code>role = admin/teller</code> |
| products           | Catalog — price is VAT-inclusive, <code>vat_rate</code> stored separately             |
| customers          | Per-business customer list                                                            |
| sales / sale_items | Sale headers and line items                                                           |
| expenses           | Business costs — visible to Admin only                                                |
| low_stock_items    | A view combining products where <code>stock &lt;= reorder_level</code>                |

Two **Edge Functions** (secure server-side code, not exposed to the browser) handle the two operations that must never be done directly from client code: - `create-business` — runs after email OTP verification, creates the business + admin profile atomically - `create-teller` — checks the caller is really an Admin, then creates the teller’s login using Supabase’s admin privileges (which are never given to the browser)

## 7. Hosting & deployment

- **Current live link:** GitHub Pages — <https://allen-otty.github.io/mini-pos/>
- Every `git push` to the `main` branch of the `mini-pos` repo automatically rebuilds this link within 1-2 minutes
- A Netlify site ( `dogo-pos` ) has also been created and is ready to go live as a second/replacement option — its first deploy needs to be triggered from a computer with the repo cloned, using the command Netlify’s own tool provided. Ask to pick this back up any time.

### To update the app in the future

```
git clone https://github.com/Allen-Otty/mini-pos.git
# make your changes to index.html / manifest.json / service-worker.js
git add -A
git commit -m "describe the change"
git push origin main
```

## 8. Known limitations & next steps

| Item               | Status                           | What’s needed                                                                                                                                                                         |
|--------------------|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Phone/SMS OTP      | Not active                       | Connect a paid SMS provider (Africa’s Talking or Twilio) to the Supabase project’s Auth settings                                                                                      |
| Email OTP template | Uses Supabase’s default template | Recommended one-time check: Supabase Dashboard → Authentication → Email Templates → “Confirm signup” → confirm it shows <code>{{ .Token }}</code> (the 6-digit code), not only a link |
| Netlify hosting    | Site created, not yet deployed   | Run the Netlify CLI deploy command from a real computer (see Section 7)                                                                                                               |
| Offline sync scope | Sales only                       | Customers/Expenses added while offline are not queued — only Sales are                                                                                                                |
| Multi-currency     | Not supported                    | App is KES-only by design (per requirements)                                                                                                                                          |
| Custom domain      | Not set up                       | Requires purchasing a domain, then pointing it at GitHub Pages or Netlify                                                                                                             |

---

## 9. Security notes

---

- The browser only ever holds a “publishable” Supabase key — safe to expose publicly, it can only do what Row Level Security policies explicitly allow
- The powerful “service role” key that can bypass all restrictions is **only** used inside the two Edge Functions, which run on Supabase’s servers, never in the browser
- Every database table has Row Level Security enabled — a Teller literally cannot query another business’s data or another business’s expenses, even by tampering with the app’s requests, because the database itself enforces it
- Passwords are handled entirely by Supabase Auth (industry-standard hashing) — this app never sees or stores raw passwords